

Week 13

Artificial Neural Networks Fundamentals

The Same Data, a New Kind of Model

Section	Question it answers
1. From Linear Models to Neural Networks	What does a neural network add to the methods we already have?
2. Building a Neural Network	How do we go from logistic regression to learned features?
3. Backpropagation	How does the network find good weights?
4. Regularisation	How do we stop the network from memorising the training data?
5. Hidden Representations	Are the network's internal features different from PCA and LDA?
6. Bringing It Together	Where does the MLP fit in the 10-method comparison?

1. From Linear Models to Neural Networks

What does a neural network add to the methods we already have?

For twelve weeks we have applied increasingly sophisticated classifiers to the same reaching dataset. Every method — logistic regression, SVM, LDA, random forests — achieved roughly 95% accuracy on direction decoding from 80 neural features. Why would we need anything else?

The answer becomes clear when you consider what those 80 features actually are. We computed the mean firing rate of each neuron over the entire trial — a single number summarising hundreds of milliseconds of neural activity. That single number discards everything about the temporal structure of the response: when the neuron starts firing, whether it ramps up gradually or bursts suddenly, whether it fires in synchrony with other neurons, whether it shows a preparatory dip before movement onset. Motor control researchers know these temporal dynamics are rich with information. Preparatory activity in premotor cortex begins hundreds of milliseconds before movement; online corrections modulate firing rates mid-reach; impairment may manifest as delayed or disorganised temporal patterns rather than changes in average rate.

Every method from Weeks 5–12 was blind to this temporal structure because we chose to summarise it away before the analysis began. The features were hand-crafted — we decided in advance that mean firing rates were the right representation. If we were wrong — if the diagnostically important signal lives in the timing, not the average — no amount of classifier sophistication could recover it.

This is what neural networks change. An ANN does not require you to design features in advance. You can feed it raw spike counts in 10 ms time bins, and the network will learn which temporal patterns are predictive. The first layer might learn to detect burst onsets; the second might learn that a burst in neuron 7 followed by a burst in neuron 34 is characteristic of 90° reaches. These are features no one designed — the network discovered them from the data.

This matters beyond temporal features. Modern neural recordings capture thousands of neurons simultaneously. You cannot hand-design features for 10,000 neurons — there are too many possible interactions. And when neural, EMG, and kinematic signals are recorded together, the question of how to combine them into a single feature vector has no obvious answer. Neural networks handle all of these naturally: they learn what to extract, what to combine, and what to ignore.

Figure 1 places this week's method in the context of the course. Each method along the spectrum applies a more complex transformation to the input, but all previous methods operated on the same hand-crafted features. The MLP is the first method that builds its own internal representation.

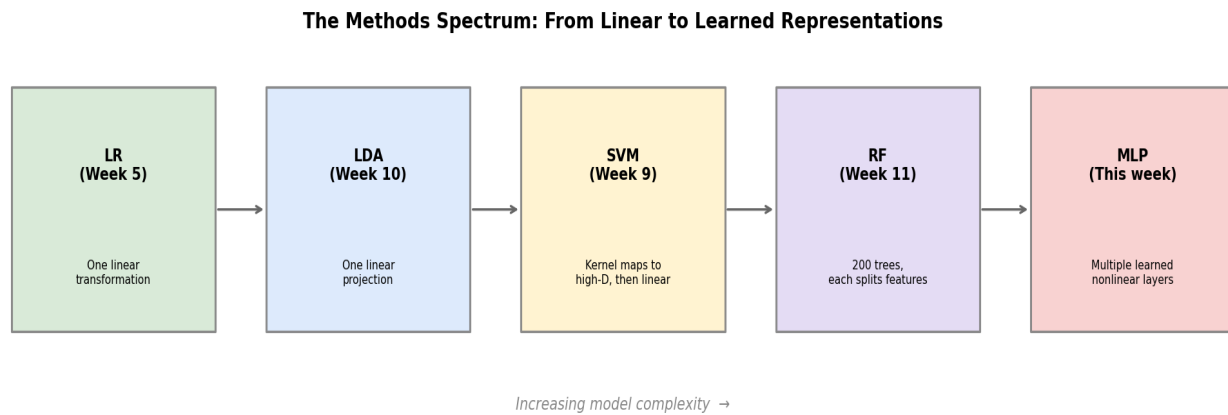


Figure 1. The methods spectrum from Weeks 5–13. All methods from LR to RF operated on the same hand-crafted features (mean firing rates). The MLP is the first to learn its own internal features through multiple nonlinear transformations.

A word of honesty: on our 480-trial dataset with well-designed features, the neural network matches but does not beat LDA or random forests (~94–95% LOSO). That is because we did hand-craft good features — cosine tuning makes mean firing rates an excellent summary of direction information. The MLP has nothing better to discover from the same inputs. The real payoff comes when the features are not so clean: raw temporal signals, high-dimensional recordings, multimodal data. Week 14 will teach the practical skills to train these networks effectively on your own data.

One note on the name. Artificial neural networks were loosely inspired by biological neurons, but modern ANNs have diverged far from biological plausibility. The name is historical, not a claim about how brains work. Motor control students should not over-interpret the biological metaphor.

2. Building a Neural Network

How do we go from a model we already know to one that learns its own features?

We will build the neural network in stages, starting from a component the class has already mastered and adding complexity only when we hit a limitation that demands it.

The building block: a single unit

The simplest possible neural network is a single unit. To see how it works, start small: classifying healthy vs impaired using just two muscles — biceps (BIC) and triceps (TRI). The unit multiplies each input by a learned weight, adds a bias, and passes the result through an activation function:

$$\text{output} = \sigma(w_{\text{BIC}} \times \text{BIC} + w_{\text{TRI}} \times \text{TRI} + b)$$

Figure 2A shows this as a diagram: inputs on the left, weights on the arrows, summation in the body, activation at the output. The equation and the diagram describe the same computation.

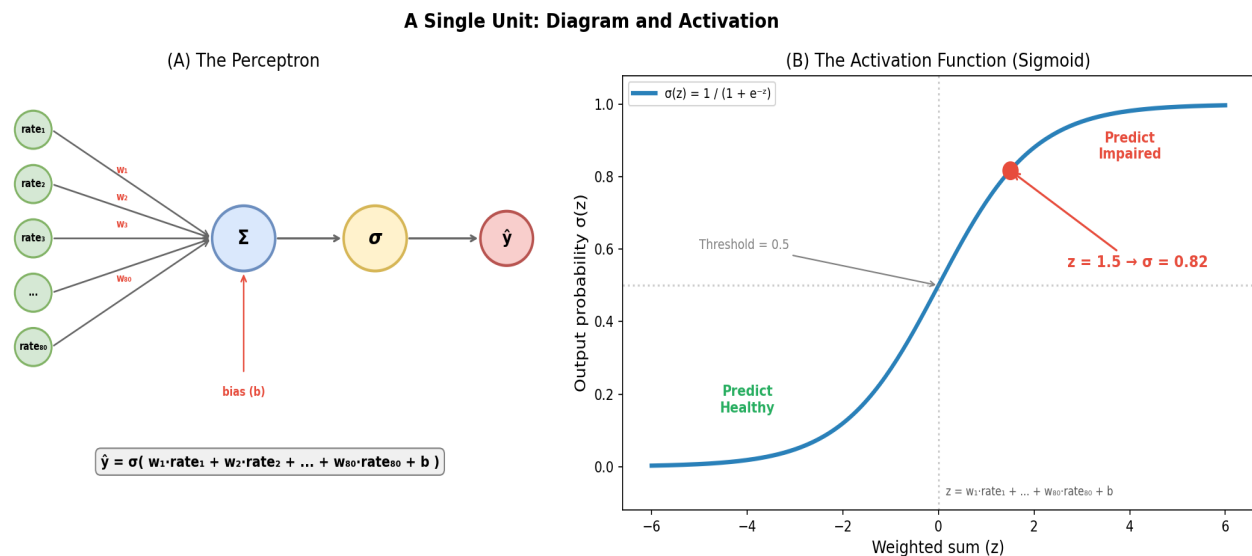


Figure 2. (A) The perceptron diagram: inputs $\times$ learned weights $\rightarrow$ sum + bias $\rightarrow$ activation $\rightarrow$ output. (B) The sigmoid activation function. The weighted sum z is converted to a probability between 0 and 1. In our worked example (red dot), a weighted sum of 1.5 becomes a probability of 0.82 — above the 0.5 threshold, so the unit predicts impaired.

To make this concrete, suppose training produces $w_{\text{BIC}} = 1.2$, $w_{\text{TRI}} = -0.8$, and $b = -0.5$. For a trial with BIC = 3.0 and TRI = 2.0: the weighted sum is $1.2 \times 3.0 + (-0.8) \times 2.0 + (-0.5) = 1.5$. Figure 2B shows what happens next: the sigmoid converts 1.5 to 0.82 (red dot), giving an 82% probability of impaired. The positive weight on BIC means higher biceps activation pushes toward impaired; the negative weight on TRI means higher triceps pushes toward healthy.

Scaling to our full dataset changes nothing fundamental — just the length of the sum. With 80 neurons instead of 2 muscles:

$$\text{output} = \sigma(w_1 \times \text{rate}_1 + w_2 \times \text{rate}_2 + \dots + w_{80} \times \text{rate}_{80} + b)$$

This is logistic regression from Week 5 — exactly the same model with new vocabulary. Weights are coefficients. Bias is intercept. Activation is sigmoid. And like logistic regression, this is supervised learning: the unit needs labelled training data (which trial was 0°, which was 180°) to

learn its weights. After Week 12's unsupervised methods, we are back in supervised territory for the rest of the course. Figure 3 confirms the connection: when there are only two well-separated classes (here, 0° vs 180° reaches), a single straight line achieves 100% accuracy. The perceptron is logistic regression, and logistic regression is enough.

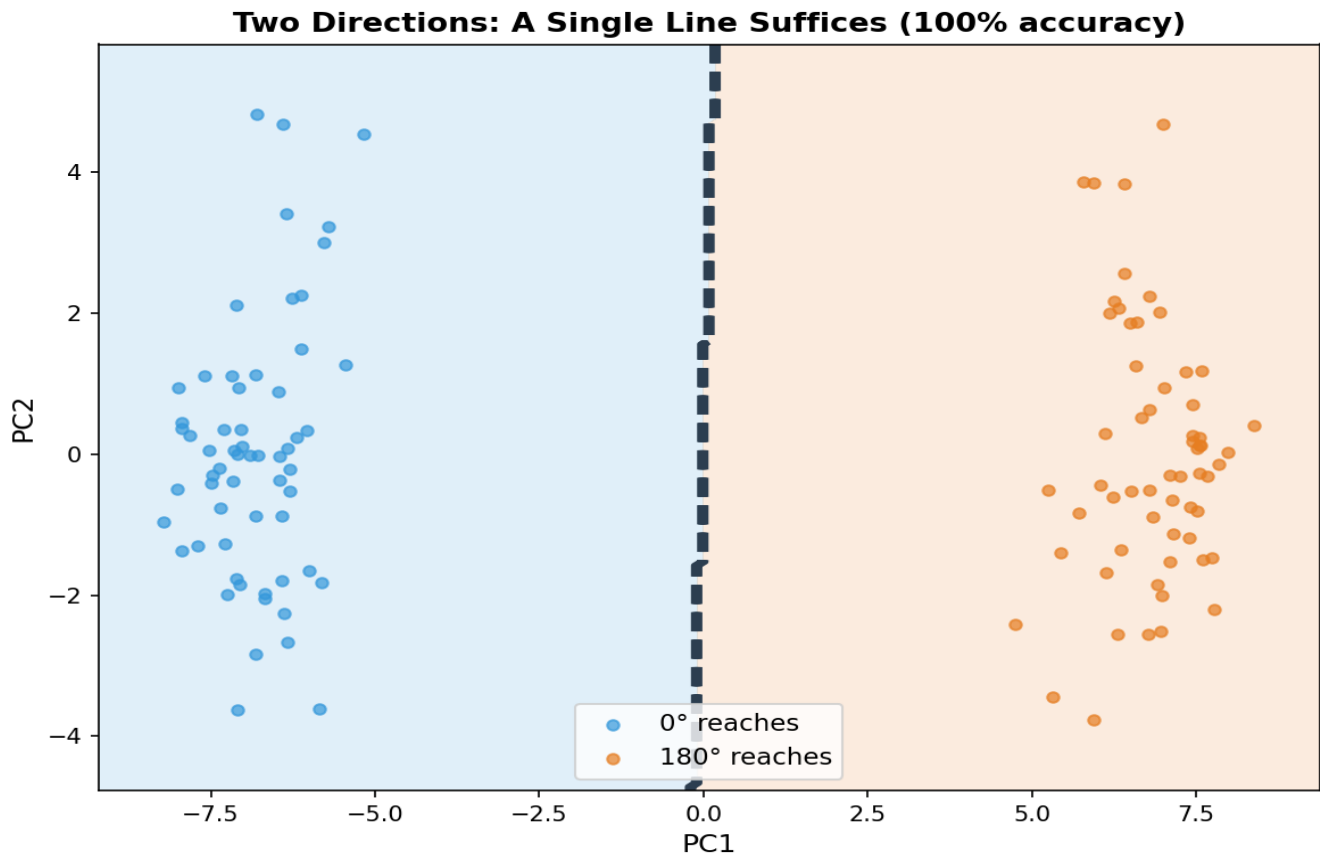


Figure 3. When a straight line suffices: 0° vs 180° reaches in PCA space. These opposite directions are well-separated by cosine tuning, and a single unit (= logistic regression) draws one line that classifies them perfectly. But what about all 8 directions?

The limitation: one line is not enough

For two opposite directions (Figure 3), a straight line suffices. But for 8-direction decoding, the directions form a ring in PCA space — no single straight line can separate 8 groups arranged in a circle. We hit this same wall with logistic regression (Week 5) and the linear SVM (Week 9). Adding more features or tuning the weights cannot help: the model's architecture limits it to linear boundaries.

The fix: hidden layers that learn new features

The breakthrough is to insert a layer of units between the inputs and the output. This hidden layer transforms the 80 firing rates into a new set of learned features before the output layer makes its decision. Here is the key: each hidden unit computes its own weighted sum of the 80 firing rates, but each one learns different weights, so each detects a different pattern.

Think of it concretely. Hidden unit 1 might learn weights that make it respond when neurons 7, 12, and 34 are all active — a co-activation pattern associated with 90° reaches. Hidden unit 2 might respond when neurons 3 and 52 are active but neuron 28 is silent — a pattern for 225° .

Each hidden unit is a learned feature detector: a combination of the original firing rates that the network has discovered is useful for classification. Nobody told the network to look for these patterns — it found them by adjusting its weights to reduce classification errors.

The output layer then classifies based on these learned features instead of the raw rates. But there is a subtlety: if each hidden unit just computes a weighted sum (like the perceptron), stacking layers would not help — a sum of sums is still a sum, and the entire network would collapse to a single linear model. The hidden units need a nonlinear activation function to break this linearity.

In Section 2 we used the sigmoid (σ), which squashes any input to a value between 0 and 1. It works, but for hidden layers a simpler function is more common: relu (rectified linear unit). Relu has a simple rule: if the weighted sum is positive, pass it through unchanged; if negative, output zero. Think of it as a neuron with a firing threshold — it either fires (positive input) or stays silent (negative input). A third option is tanh, a smooth S-curve between -1 and $+1$. On our data all three give similar accuracy ($\sim 95\%$), so we use relu for its simplicity. Figure 4 shows all three side by side.

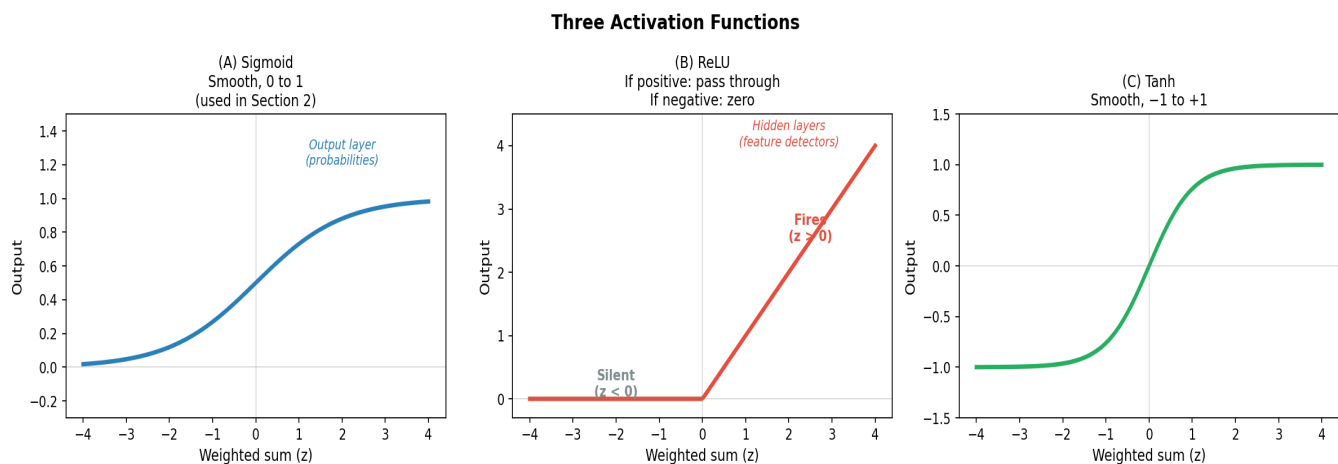


Figure 4. Three activation functions. (A) Sigmoid (from Section 2): smooth curve between 0 and 1, used at the output layer to produce probabilities. (B) ReLU: if positive, pass through; if negative, output zero — the default for hidden layers. (C) Tanh: smooth curve between -1 and $+1$. On our data, all three produce similar accuracy.

With relu activations, each hidden unit outputs either zero (the pattern it detects is absent) or a positive value (the pattern is present, and stronger activations mean stronger matches). The output layer can now draw complex boundaries in the space of these learned features, even though each individual unit still computes a simple weighted sum.

Adding depth: features of features

A second hidden layer takes this further. The first layer learns simple patterns (which neurons are co-active). The second layer learns combinations of those patterns — for example, that the 90° -associated pattern from hidden unit 1 tends to co-occur with a specific timing pattern from hidden unit 5 during fast reaches. This hierarchy of increasingly abstract features is what distinguishes neural networks from every method we have seen so far: LDA finds one set of axes, RF splits on one feature at a time, but an MLP builds a layered representation where each layer's features are richer than the last.

Our architecture

For the 8-direction task we use two hidden layers: 80 inputs $\rightarrow$ 64 hidden units (relu) $\rightarrow$ 32 hidden units (relu) $\rightarrow$ 8 output units (one per direction). Figure 5 shows the structure. The total parameter count is 7,528 — more than LDA (560) but fewer than RF ($\sim 40,000$).

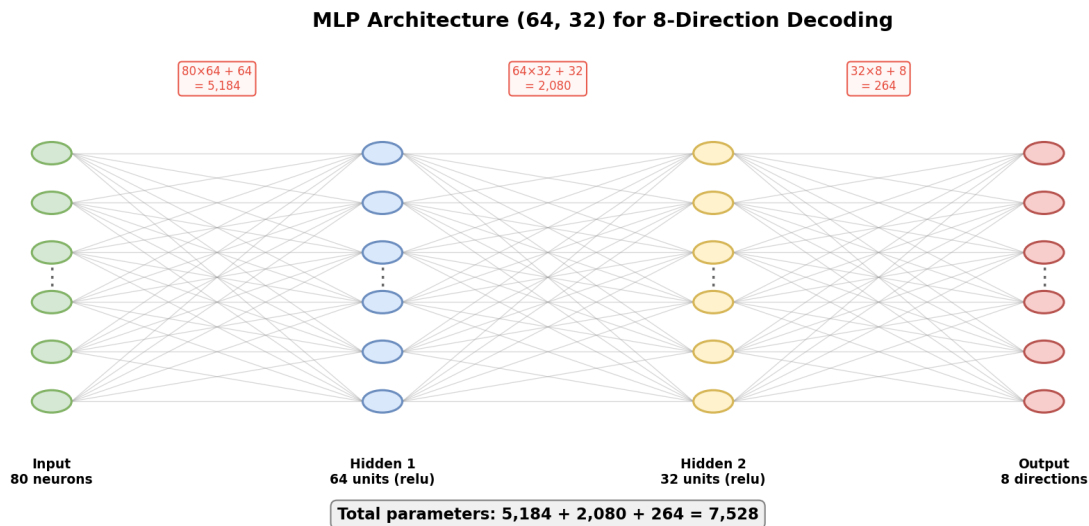


Figure 5. MLP architecture (64, 32) for 8-direction decoding. Lines between layers represent learned weights. Red annotations show parameter counts: $80 \times 64 + 64$ biases = 5,184 in the first layer, totalling 7,528. Compare with LDA (560) and RF ($\sim 40,000$).

How important is the choice of architecture? Figure 6 sweeps from a single layer of 8 units to three layers of (128, 64, 32). All configurations achieve 94–95%. On 480 trials with well-designed cosine-tuning features, architecture barely matters — the data is too small and the features too clean. This is an important lesson: on small datasets, having enough data matters more than architectural choices.

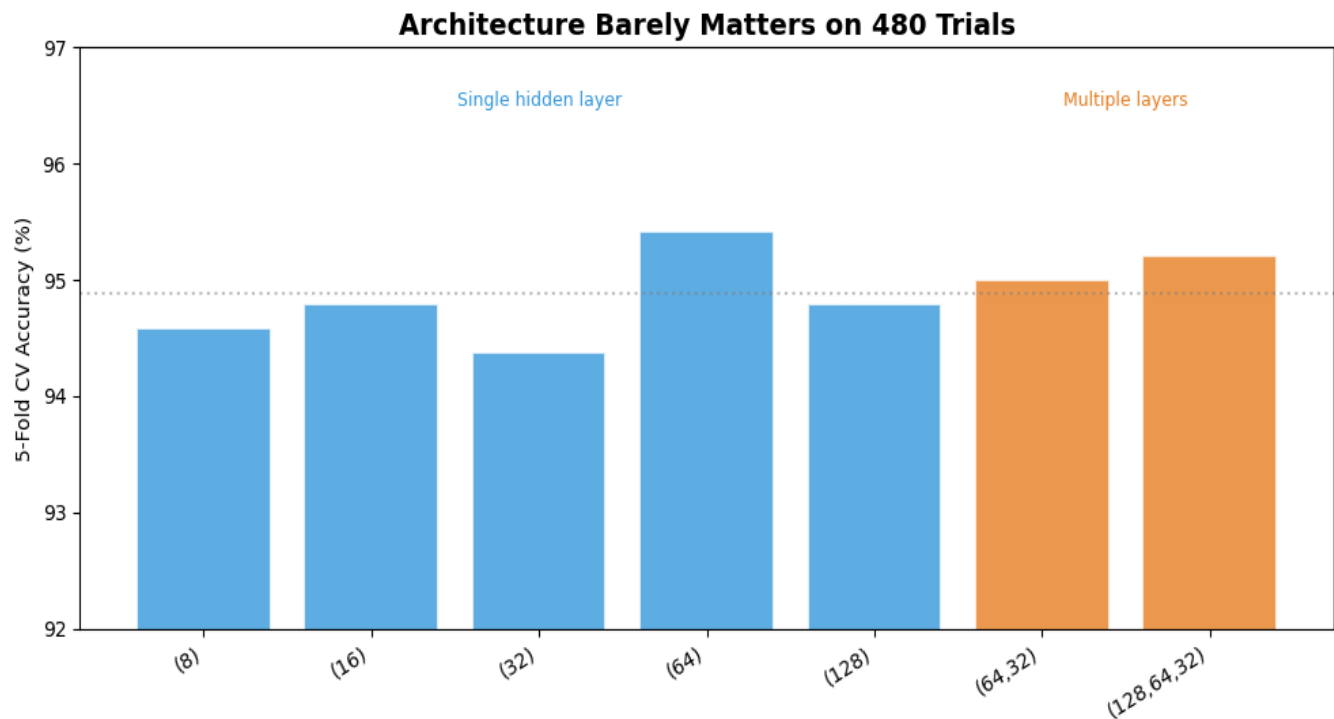


Figure 6. Architecture sweep (5-fold CV, neural 8-direction). Accuracy is flat across all configurations. Blue: single hidden layer. Orange: multiple layers. On this dataset, architecture does not matter.

Although accuracy is similar across architectures on the standard 8-direction task (LDA achieved 95.4% in Week 10), the real power of hidden layers becomes visible when we pose a problem that is fundamentally nonlinear. Suppose we group the 8 directions into two classes: cardinal (0° , 90° , 180° , 270°) vs diagonal (45° , 135° , 225° , 315°). On the original task, LDA succeeded because each direction sat in its own region of space and straight boundaries between them sufficed. But cardinal and diagonal directions alternate around the ring — no straight line can separate them, just as you cannot separate black and white squares on a checkerboard with a single cut.

Figure 7 shows what happens. LDA draws one straight line and achieves 50% — pure chance. It literally cannot solve this problem. The single-layer MLP (64 units) carves the space into alternating regions and achieves 96%. The two-layer MLP (64, 32) produces even more complex boundaries. This is what hidden layers do: they bend the decision surface into shapes that linear methods cannot create.

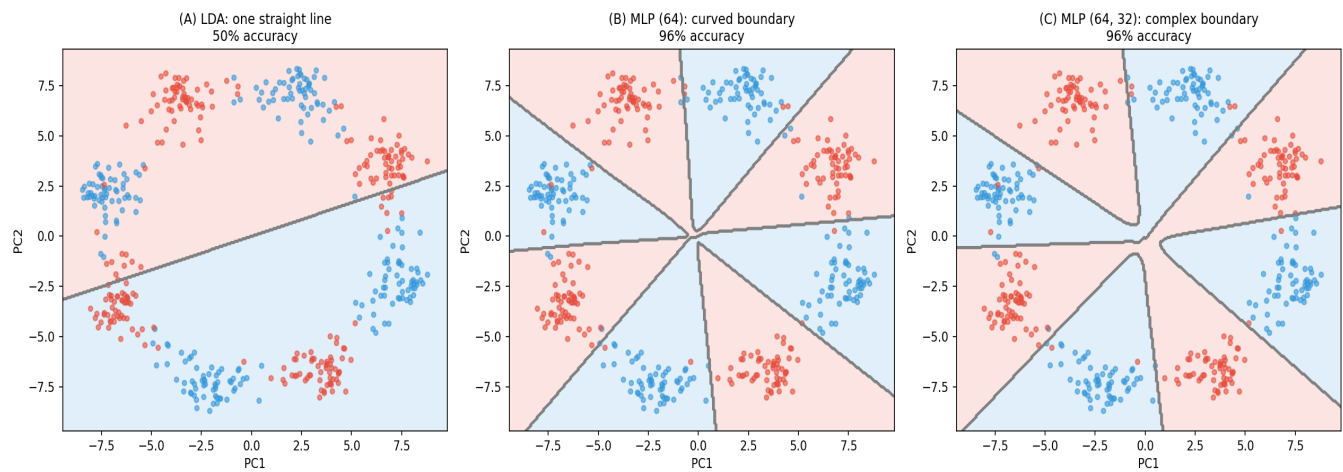
A Problem LDA Cannot Solve: Cardinal vs Diagonal Directions

Figure 7. A problem LDA cannot solve. Cardinal directions (0°, 90°, 180°, 270°, blue) alternate with diagonal directions (45°, 135°, 225°, 315°, red) around the ring. (A) LDA: one straight line, 50% accuracy (chance). (B) MLP (64): curved boundaries separate the alternating groups (96%). (C) MLP (64, 32): more complex boundaries, same accuracy. Hidden layers create decision surfaces that linear methods cannot.

3. Backpropagation — How the Network Learns

How does the network find good weights?

The MLP has 7,528 weights that need to be set to good values. The training process works as follows. First, the network makes a prediction for each training trial by passing the firing rates forward through the layers — this is called the forward pass. Then it compares the prediction with the true direction label using a loss function (cross-entropy, the same function logistic regression used in Week 5, extended to 8 reach directions). The loss measures how wrong the prediction was.

Next comes the key step: backpropagation. The network works backward through the layers, asking at each weight: “how much did this weight contribute to the error?” Weights that contributed more to the error are adjusted more. The adjustment is proportional to the learning rate, which controls how big each step is — too large and the network overshoots, too small and it converges slowly.

Figure 8 traces this process through a tiny example. In Panel A, two firing rates flow forward through the network: the weighted sums produce activations of 2.1 (hidden unit fires) and 0.0 (hidden unit stays silent because relu zeroed a negative sum). The output is 0.73, and the network predicts 45°. Panel B reveals the error: the true direction was 90°. Panel C shows backpropagation in action: the error signal flows backward through the network, and each weight receives an adjustment (Δw) proportional to how much it contributed to the mistake. Notice that the silent hidden unit (0.0 in the forward pass) receives no adjustment — it did not participate in the prediction, so it bears no responsibility for the error.

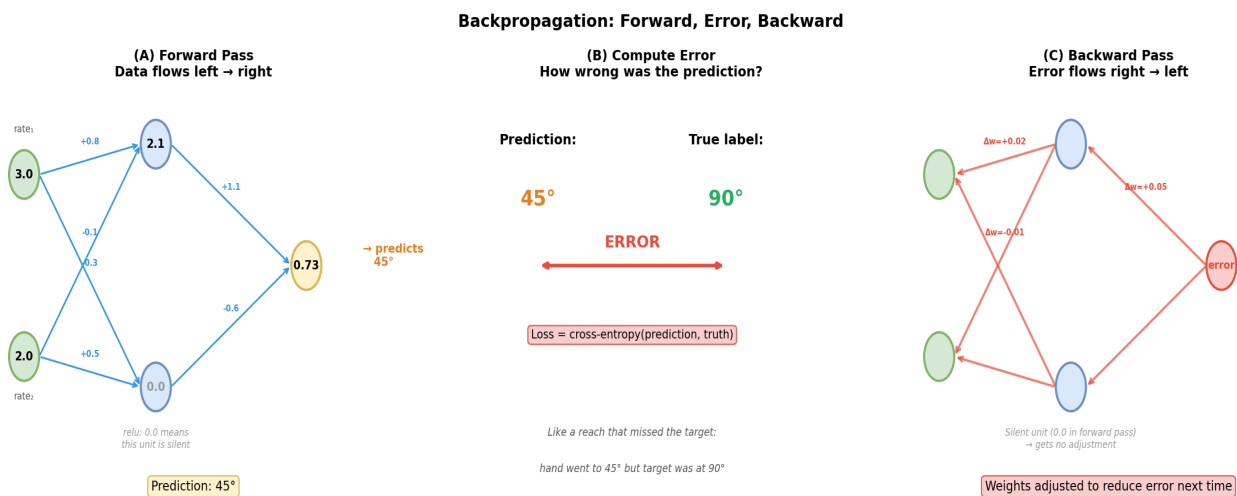


Figure 8. Backpropagation in three steps. (A) Forward pass: firing rates flow left to right through weights and activations to produce a prediction (45°). (B) Error: the prediction is compared with the true label (90°) — like a reach that missed the target. (C) Backward pass: the error signal flows right to left, adjusting each weight to reduce the error next time. The silent unit (0.0) gets no adjustment.

The analogy to motor learning is precise: a person reaching toward a target observes the error (hand went to 45°, target was at 90°) and the motor system adjusts the command for the next attempt. The network does the same thing — error signals propagate backward through a hierarchy to update the parameters that caused the error.

A subtlety worth noting: backpropagation computes the gradients — it tells us which direction to adjust each weight. But how far to step in that direction is decided by the optimizer. The default (Adam) adapts the step size for each weight automatically. Other optimizers take fixed steps, or use momentum to avoid getting stuck. Week 14 will cover these choices in detail; for now, the key idea is that backpropagation is always the engine that computes the error signals, and the optimizer is the steering wheel that decides what to do with them.

This process repeats for many epochs (passes through the entire training set). Figure 10 shows what happens over time. Training accuracy climbs to 100% by epoch 100 — the network has memorised every training trial. But test accuracy plateaus at about 96.5% and stops improving. This is the same overfitting pattern we saw with decision trees in Week 11 (Figure 2B): too many parameters (7,528) for too few trials (480).

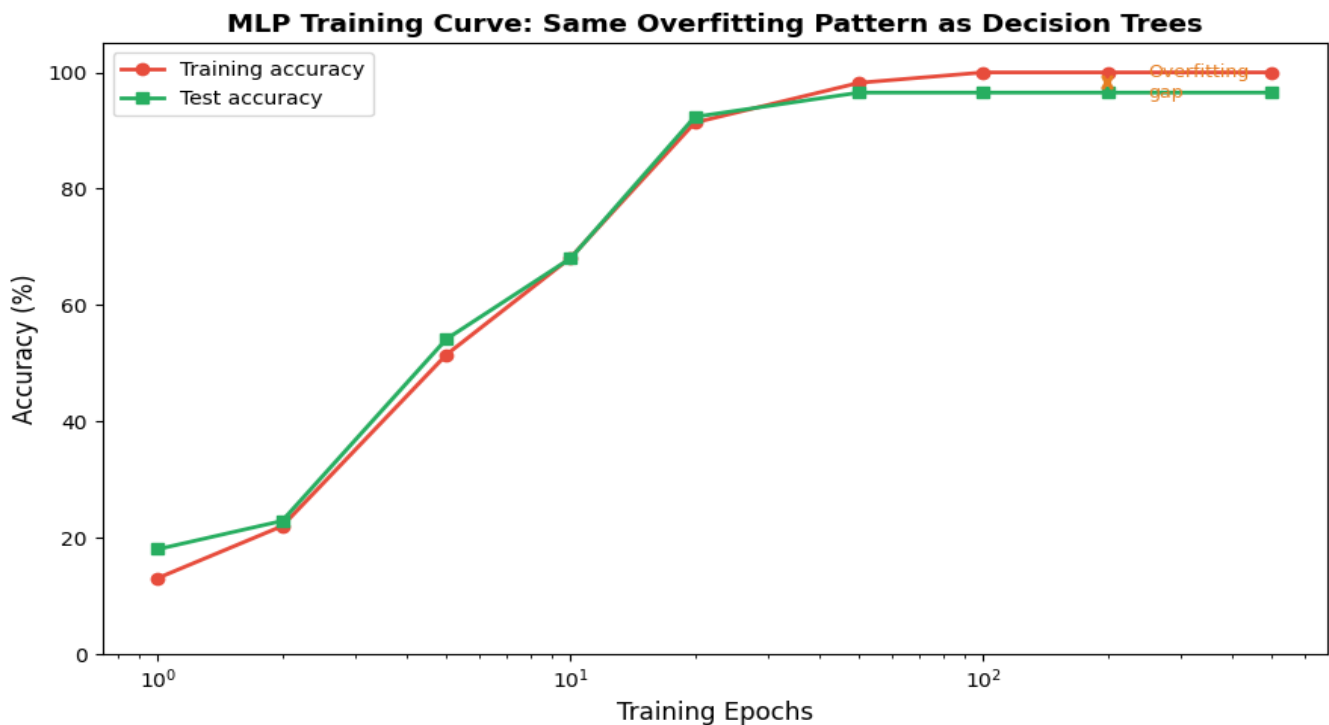


Figure 9. MLP training curve (64, 32 architecture, 70/30 train/test split). Training accuracy (red) reaches 100% by epoch 100. Test accuracy (green) plateaus at ~96.5%. The orange arrow marks the overfitting gap — the same pattern as decision tree depth in Week 11, Figure 2.

4. Regularisation — Preventing Overfitting

How do we stop the network from memorising the training data?

With 7,528 parameters and only 480 trials, the network can easily memorise every training example (as Figure 9 showed). Three strategies address this.

L2 regularisation (weight decay) penalises large weights by adding their squared magnitude to the loss function. This encourages the network to use small, distributed weights rather than relying on a few large ones. It is the same idea as the C parameter in SVM (Week 9): smaller C meant stronger regularisation. In the MLP, the hyperparameter that controls the strength of L2 regularisation is called alpha — larger alpha means heavier penalisation of large weights.

Early stopping monitors the test loss during training and stops when it starts increasing. This is analogous to choosing the optimal tree depth in Week 11 — stop before the model memorises.

Dropout randomly sets a fraction of hidden units to zero during each training step, forcing the network not to rely on any single unit. This is conceptually similar to random feature subsets in random forests (Week 11, Section 2): decorrelating the learned features so that the ensemble (or in this case, the surviving units) is more robust.

Figure 10 tests L2 regularisation on our data. The result is striking: accuracy is flat across five orders of magnitude of alpha (0.0001 to 1.0). Regularisation barely matters here because the bottleneck is data size, not model flexibility. With only 480 trials, even an unregularised network cannot find enough spurious patterns to significantly hurt test performance. On larger datasets, regularisation becomes essential.

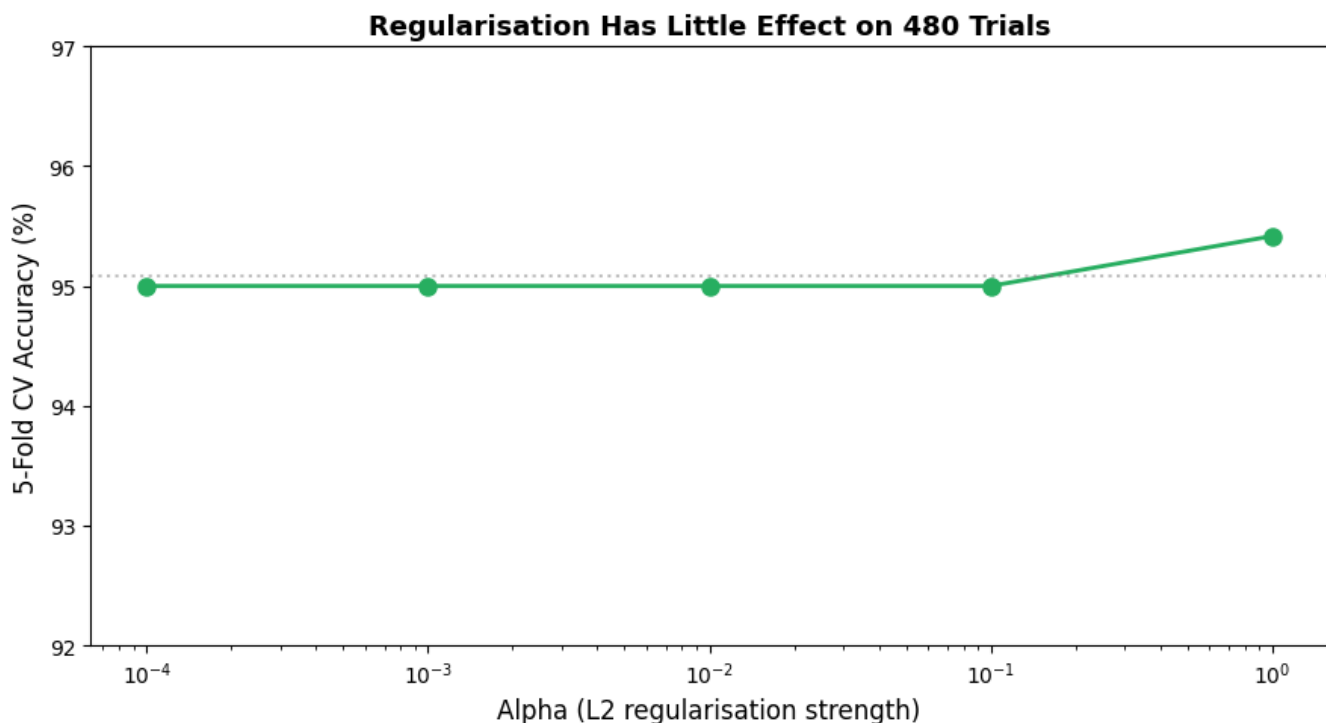


Figure 10. L2 regularisation (alpha) vs accuracy (5-fold CV, neural 8-direction). The curve is flat — regularisation strength has almost no effect on this small dataset. Compare with SVM's sharp sensitivity to $C \times \gamma$ (Week 9) and RF's similarly flat hyperparameter curves (Week 11).

When would regularisation become essential? Consider three changes to our setup. First, more features: if instead of 80 mean firing rates we fed the network raw time-series — 80 neurons $\times$ 50 time bins = 4,000 inputs — the network would have far more opportunity to memorise noise, and without regularisation, test accuracy would collapse. Second, more classes or subtler distinctions: if we asked the network to decode not just 8 directions but also 3 speeds and 2 impairment levels (48 classes from 480 trials), the ratio of parameters to training examples per class would become dangerous, and dropout would become critical for preventing the network from memorising the few examples per class. Third, noisier or more variable data: our cosine-tuned neurons produce clean, consistent firing rate patterns. Real cortical recordings contain trial-to-trial variability, electrode drift, and movement artefacts that create spurious patterns for the network to latch onto — exactly the situation where early stopping and weight decay earn their keep. In short, regularisation matters when the gap between what the network could memorise and what it should learn is large. On our small, clean dataset, that gap is narrow.

5. Hidden Representations — What the Network Learned

Are the network's internal features different from PCA and LDA?

If the MLP's accuracy matches LDA and RF, what is the point? The answer lies in what the network learns internally. After training, the 32-dimensional output of the second hidden layer is the network's learned feature space — 80 firing rates compressed to 32 numbers that the output layer uses for classification. We can extract this representation and compare it with the representations from PCA and LDA.

Figure 11 projects all three representations to 2D for visualisation. The key metric is the silhouette score (introduced in Week 12 for evaluating clustering). While accuracy measures how often the classifier gets the right answer on held-out data, silhouette measures something different: how well-separated the clusters are in the feature space before any classification is applied. A high silhouette means the 8 direction groups form tight, well-separated clusters; a low silhouette means they overlap. Two methods can have similar accuracy but very different silhouettes if one creates a cleaner internal geometry.

PCA gives a silhouette of 0.440, LDA gives 0.456, and the MLP's hidden layer gives 0.621 — 37% higher than LDA. The network has learned a representation where the 8 directions are more cleanly separated than in either linear projection, even though the final classification accuracy is similar.

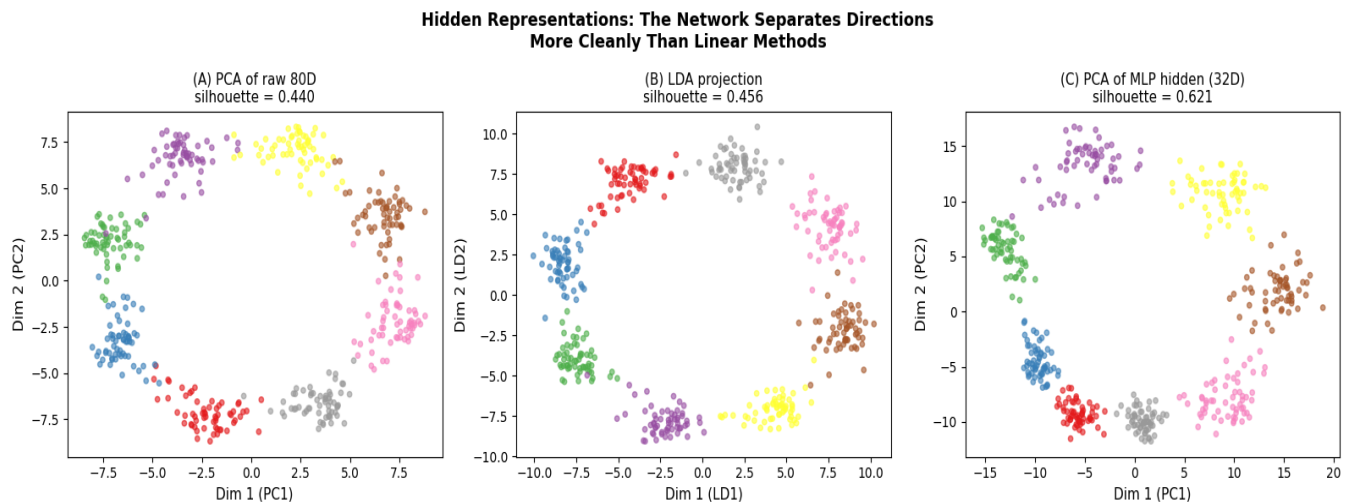


Figure 11. Hidden representations compared. (A) PCA projection of raw 80D data (silhouette=0.440). (B) LDA projection (silhouette=0.456). (C) PCA projection of the MLP's 32D hidden layer (silhouette=0.621). The network separates directions more cleanly than either linear method.

Why does this matter if accuracy is the same? On our small, clean dataset, all methods are already near ceiling (~95%). The richer representation has nowhere to show its advantage. But on larger, noisier datasets where features are not perfectly hand-crafted, learning better-separated representations translates directly to better classification. Week 14 will teach the practical training skills — PyTorch, learning rates, data augmentation, saliency maps — needed to apply these networks to your own research data.

One additional observation: only 42.9% of hidden units are non-zero for a given trial (because relu zeroes out negative inputs). The network automatically selects which internal features to activate

for each trial — a learned form of feature selection, different from the explicit feature importance ranking that random forests provided in Week 11.

6. Bringing It Together

Where does the MLP fit in the 10-method comparison?

Figure 12 places the MLP alongside all nine methods from Weeks 5–12 on the neural 8-direction task. The MLP achieves 94.2% LOSO accuracy — comparable to LDA (95.4%) and RF (95.2%), but with more hyperparameters to tune and longer training time. The new column, “Learns Features?,” highlights the MLP’s unique property: it is the only method that builds its own internal representation rather than operating on features we designed.

The Complete Picture: 10 Methods on Neural 8-Direction

Method	Type	Neural Dir.	Params	Needs Tuning?	Interpretability	Learns Features?
NB	Supervised	95.2%	~1K	No	Priors	No
LR	Supervised	91.2%	640	Yes (C)	Coefficients	No
KNN	Supervised	94.6%	—	Yes (k)	None	No
Lin SVM	Supervised	92.7%	~600	Yes (C)	Weights	No
RBF SVM	Supervised	94.6%	~2K	Yes (C, γ)	None	No
LDA	Supervised	95.4%	560	No	Loadings	No
RF	Supervised	95.2%	~40K	No	Feature imp.	No
K-means	Unsupervised	ARI=0.92	640	Yes (k)	Centroids	No
GMM	Unsupervised	ARI=0.92	~1K	Yes (k)	Means+cov	No
MLP	Supervised	94.2%	7,528	Yes	Hidden reps	Yes

Figure 12. The complete 10-method comparison on neural 8-direction. Supervised methods report LOSO accuracy; unsupervised methods (green) report ARI. The MLP (red row) matches classical methods on accuracy but is the only method that learns its own features.

One more comparison, continuing the electrode drift analysis from Weeks 9–11. Figure 13 adds the MLP to the drift robustness test: Gaussian noise with increasing σ is added to the 80 neural features, and accuracy is measured at each noise level. The MLP (red, thick line) degrades more slowly than most methods — at $\sigma=5$, it still achieves 95.0% while RF has dropped to 92.7% and LR to 87.5%. Why? Each hidden unit combines many neurons into a single learned feature. If one neuron drifts, the other neurons in that combination compensate — the distributed representation provides natural robustness. This is relevant for BCI applications where electrode stability is a concern.

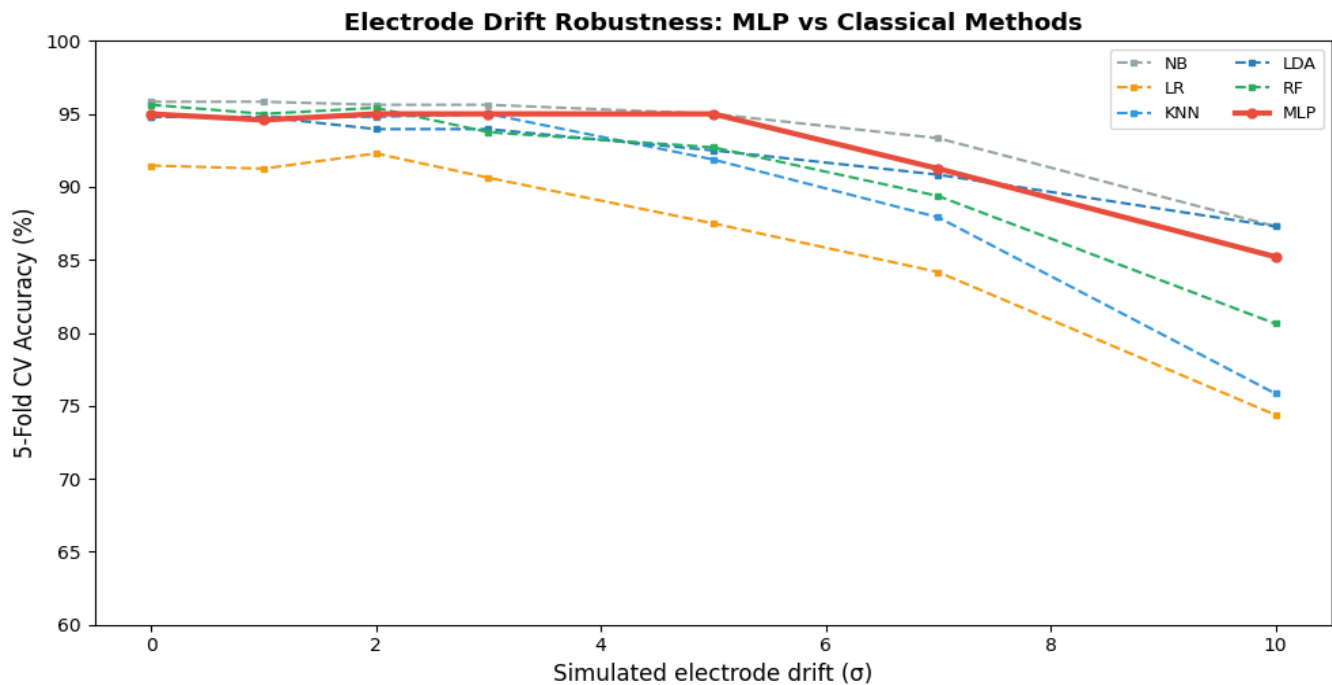


Figure 13. Electrode drift robustness (5-fold CV, neural 8-direction). The MLP (red, thick) degrades more slowly than RF and LR at moderate drift ($\sigma=5$), though NB and LDA show comparable robustness. The MLP's distributed hidden representations provide natural tolerance to individual neuron drift.

When to use MLPs:

- When you have enough data. Our 480 trials are too few for the MLP to outperform LDA. With thousands of trials, the MLP's learned features begin to matter.
- When the right features are not obvious. If you do not know whether to use mean firing rates, peak rates, burst counts, or something else, the network can discover what works.
- When you plan to extend to deeper architectures. The MLP is the foundation for recurrent and convolutional networks (covered in the Week 15 deep learning handout).

When NOT to use MLPs:

- When data is small and features are well-designed. On 480 trials with cosine-tuned firing rates, LDA or RF will match or beat the MLP with less effort.
- When interpretability is essential. A clinician can inspect LDA loadings or RF feature importance directly. The MLP's 7,528 weights are much harder to interpret.
- When computational resources or time are limited. Training an MLP takes seconds to minutes; fitting LDA takes milliseconds.

What we learned this week

- A perceptron is logistic regression with a new name. Stacking layers with nonlinear activations adds the ability to learn features automatically.
- Backpropagation finds good weights by propagating error signals backward through the layers — analogous to sensory error correction in motor learning.
- On our 480-trial dataset, the MLP matches but does not beat classical methods (~94–95%).

- The network's hidden representations separate directions more cleanly (silhouette 0.621) than PCA (0.440) or LDA (0.456) — richer features, same accuracy.
- Regularisation matters less than data size on small datasets. Week 14 will teach the practical skills to train and deploy these networks on your own research data.

References

Rosenblatt F. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408, 1958.

Rumelhart DE, Hinton GE, Williams RJ. Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536, 1986.

Goodfellow I, Bengio Y, Courville A. *Deep Learning*. MIT Press, 2016.

Glaser JI, Benjamin AS, Farhoodi R, Kording KP. The roles of supervised machine learning in systems neuroscience. *Progress in Neurobiology*, 175, 126–137, 2019.

Georgopoulos AP, Kalaska JF, Caminiti R, Massey JT. On the relations between the direction of two-dimensional arm movements and cell discharge in primate motor cortex. *Journal of Neuroscience*, 2(11), 1527–1537, 1982.